Stochastic Processes

Semester: 14042

Lecturer: Dr. Peyvandi

Project Report I

Students: Mobin Yousefi, Mehrshad Haghighat IDs: 402100594, 402100418

Introduction

This report walks through what we did in each part of our final project for the Stochastic Processes course. The project is split into four tasks: building Gaussian Process (GP) priors from different covariance kernels, doing GP regression on noisy data, running MCMC to learn a kernel hyperparameter, and finally implementing Langevin dynamics as an alternative sampling method. Below we explain the goal of each part, what we actually implemented, and what the results looked like.

1 Task 1 — GP Priors: RBF and Brownian Motion Kernels

The first thing we needed to understand was that a Gaussian Process is basically a probability distribution over *functions*, defined entirely by a mean function (which we set to zero everywhere) and a covariance function, i.e. a kernel $k(x, x')$.

We implemented two kernels:

- **RBF (squared-exponential) kernel:**

$$k(x, x') = \sigma^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$

  We computed this efficiently over a whole grid of points at once using the identity $\|x_i - x_j\|^2 = \|x_i\|^2 + \|x_j\|^2 - 2x_i x_j$, instead of looping over every pair.

- **Brownian motion (Wiener process) kernel:**

$$k(s, t) = \min(s, t)$$

  This comes directly from the definition of Brownian motion: since increments are independent and $W(0) = 0$, the covariance between $W(s)$ and $W(t)$ for $s < t$ works out to just $\mathrm{Var}(W(s)) = s$.

To actually draw sample functions from each prior, we built the full covariance matrix over a grid (200 points for RBF on $[-5, 5]$, 500 points for Brownian motion on $[0, 10]$), added a tiny jitter term (10^{-8}) to the diagonal for numerical stability, and used `np.random.multivariate_normal` to draw 5 sample paths from each.

What we observed: the RBF samples are very smooth — they look like nice wavy curves with no sharp corners, which makes sense since the RBF kernel is infinitely differentiable. The

Brownian motion samples, on the other hand, look jagged and rough, which reflects the fact that Brownian paths are continuous everywhere but differentiable nowhere.

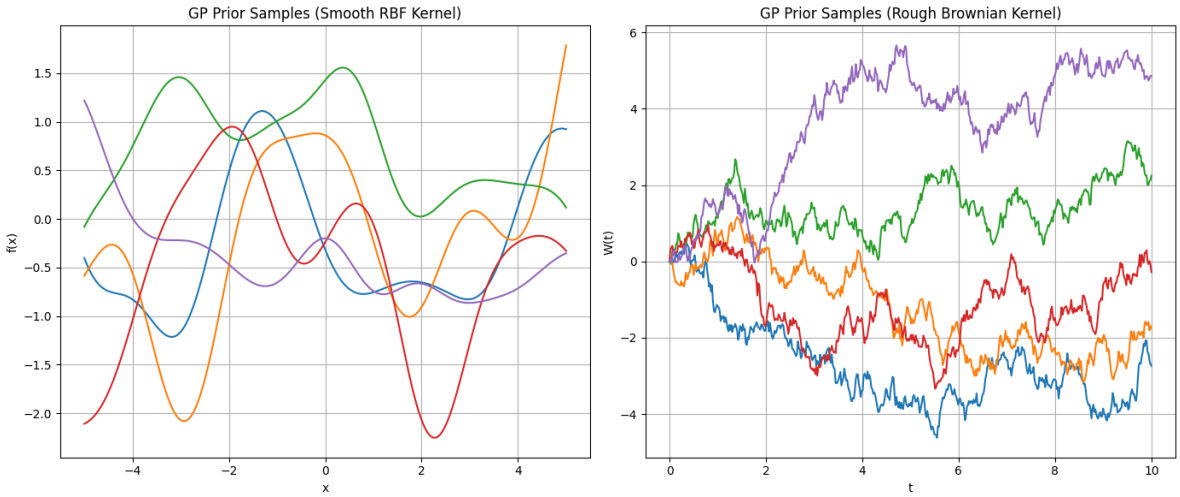

Figure 1: Sample paths drawn from the GP prior under the RBF kernel (left) and the Brownian motion kernel (right).

2 Task 2 — GP Regression

For this part we moved from just sampling priors to actually doing regression: given a handful of noisy observations, predict the function everywhere else, with uncertainty.

We generated training data from $y_i = \sin(x_i) + \epsilon_i$, with $\epsilon_i \sim \mathcal{N}(0, 0.2^2)$, at nine points between -4 and 4.

The key idea is that if we assume a GP prior on f, then the joint distribution of the training outputs and the values we want to predict is jointly Gaussian, so we can condition on the observed data to get a closed-form posterior:

$$\mu_* = K_{*n}(K_{nn} + \sigma_n^2 I)^{-1} y, \qquad \Sigma_* = K_{**} - K_{*n}(K_{nn} + \sigma_n^2 I)^{-1} K_{n*}$$

We built K_{nn} (training–training), K_{*n} (test–training), and K_{**} (test–test) using the RBF kernel, added the observation noise variance to the diagonal of K_{nn}, and computed the posterior mean and covariance with a matrix inverse (the assignment allowed `np.linalg.inv` directly, though in a real implementation Cholesky-based solves are preferred for numerical stability).

What we observed: the posterior mean tracks the sine wave nicely near the training points, and the 95% confidence band (computed as $\mu_* \pm 2\sqrt{\text{diag}(\Sigma_*)}$) is narrow right at the observed points and widens noticeably in the gaps between them — which is exactly the behavior you'd expect: the model is confident where it has seen data and uncertain where it hasn't.

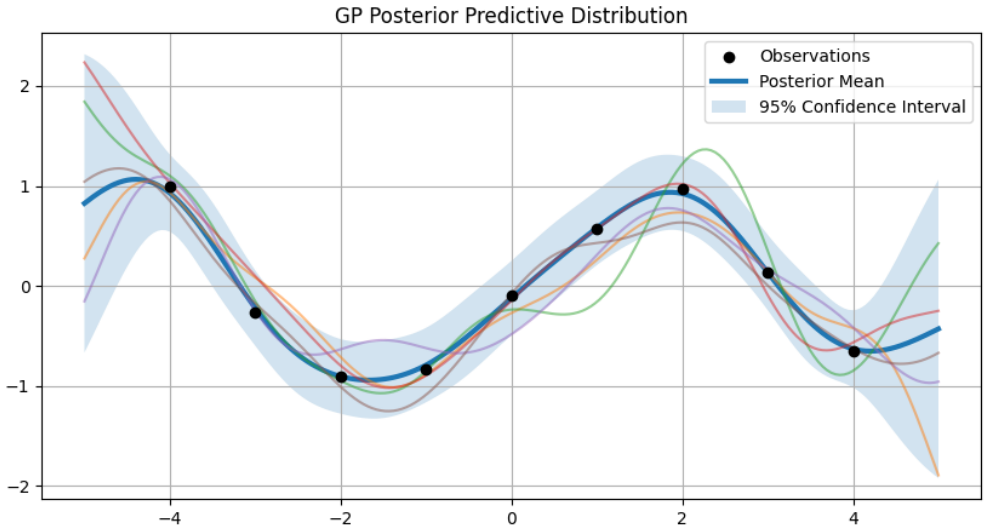

Figure 2: GP posterior mean, 95% confidence band, and posterior sample paths, together with the noisy training observations.

3 Task 3 — MCMC for the Length-Scale Hyperparameter

In Task 2 we just fixed the length-scale to $\ell = 1$. Here, we wanted to actually infer it from the data using Bayesian inference, since in practice you rarely know the "right" length-scale in advance.

We put a Gamma$(2, 1)$ prior on ℓ (restricted to $\ell > 0$), and used the GP marginal likelihood as the likelihood term:

$$\log p(y \mid X, \ell) = -\tfrac{1}{2} y^\top K_\ell^{-1} y - \tfrac{1}{2} \log |K_\ell| - \tfrac{n}{2} \log(2\pi)$$

where K_ℓ is the noisy RBF covariance matrix built with length-scale ℓ. Adding the log-prior gives the (unnormalized) log-posterior.

Since we can't sample directly from this posterior, we implemented a **Metropolis-Hastings** sampler:

1. Propose $\ell' = \ell_{\text{current}} + \mathcal{N}(0, 0.15^2)$ (a symmetric random-walk proposal).
2. Compute the log-acceptance ratio $\log \alpha = \log p(\ell' \mid y) - \log p(\ell_{\text{current}} \mid y)$.
3. Accept the proposal if $\log(U) < \log \alpha$ for $U \sim \text{Uniform}(0, 1)$, otherwise stay put.

We ran this for 15,000 iterations, then discarded the first 20% as burn-in.

What we observed: the trace plot mixes reasonably well around a stable value rather than drifting or getting stuck, and the resulting histogram gives a nice bell-shaped approximation of the posterior over ℓ, letting us read off a plausible range for the length-scale directly from the data instead of guessing it.

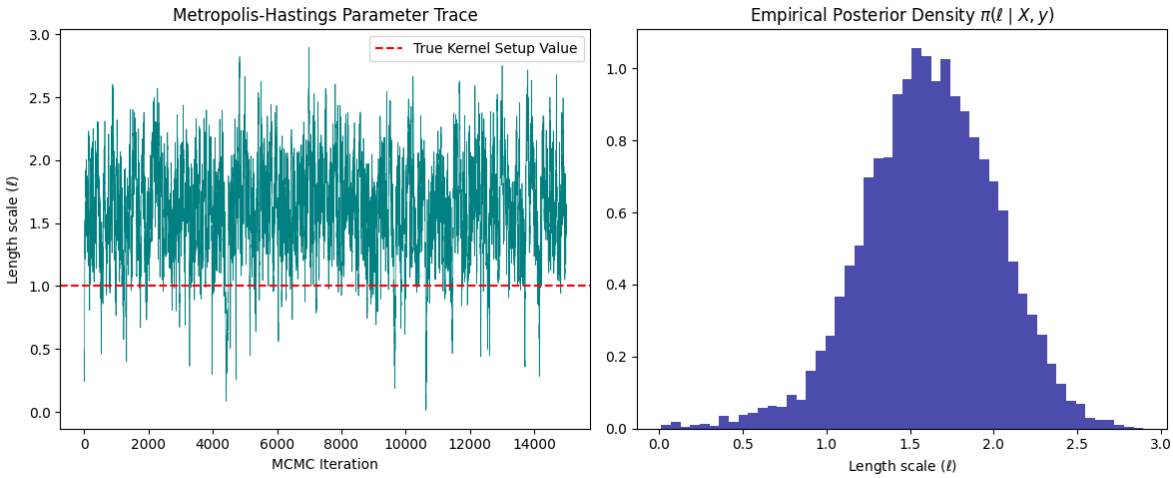

Figure 3: Left: Metropolis-Hastings trace of the length-scale parameter. Right: resulting empirical posterior density after burn-in.

4 Task 4 — Langevin Dynamics

The last part explores a completely different way of sampling from a distribution: instead of an accept/reject MCMC scheme, we used a stochastic differential equation (SDE) whose stationary distribution is the target density.

The target here is a symmetric two-component Gaussian mixture, $\pi(x) \propto \frac{1}{2}e^{-\frac{1}{2}(x+2)^2} + \frac{1}{2}e^{-\frac{1}{2}(x-2)^2}$. The Langevin SDE is

$$dX_t = \tfrac{1}{2}\nabla \log \pi(X_t)\, dt + dW_t,$$

which has π as its stationary distribution. We approximated the score $\nabla \log \pi(x)$ numerically with a central difference, and discretized the SDE with the **Euler–Maruyama** scheme:

$$X_{t+\epsilon} = X_t + \tfrac{\epsilon}{2}\nabla \log \pi(X_t) + \sqrt{\epsilon}\, Z, \qquad Z \sim \mathcal{N}(0,1)$$

using step size $\epsilon = 0.02$ for 20,000 steps starting from $x_0 = 0$. Note the noise term scales with $\sqrt{\epsilon}$, not ϵ — this comes directly from the Brownian motion increment variance being ϵ, so its standard deviation is $\sqrt{\epsilon}$.

What we observed: the trajectory bounces around and (after some time) visits both modes of the target distribution, and the histogram of samples (after discarding an initial burn-in of 5,000 steps) lines up very well with the true bimodal density plotted on top of it. This confirmed that simulating the SDE really does recover the target distribution, without ever needing an accept/reject step.

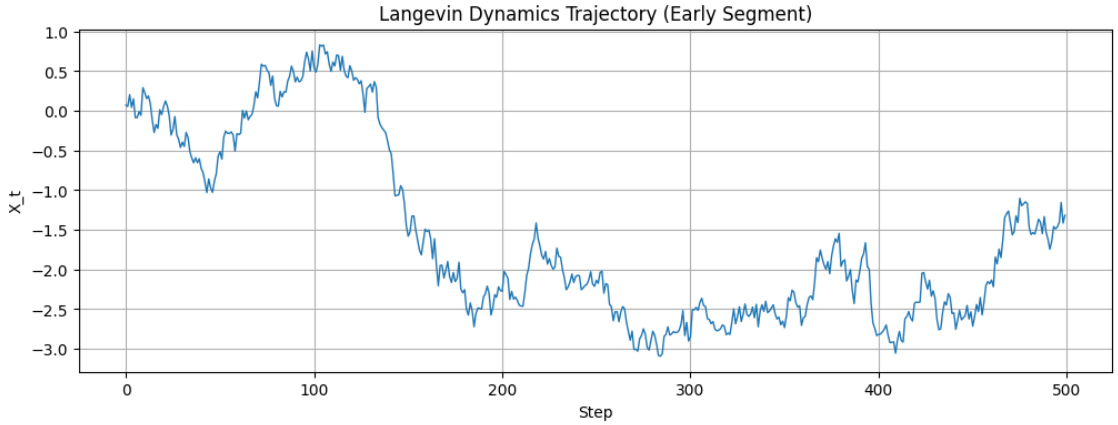

Figure 4: Early segment of the Langevin trajectory.

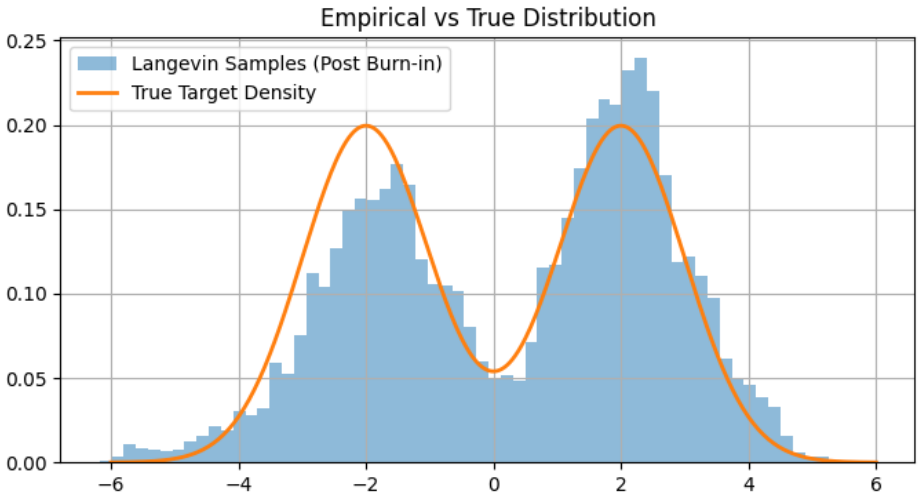

Figure 5: Empirical distribution of Langevin samples (post burn-in) versus the true target density.

Conclusion

Overall this project tied together a lot of ideas from the course into one coherent story: Gaussian Processes and their kernels (Task 1), using GPs for Bayesian regression (Task 2), learning a GP hyperparameter with MCMC (Task 3), and using an SDE-based sampler as an alternative to standard MCMC (Task 4). What we found most interesting is that Brownian motion shows up twice in very different roles — first as one of the kernels for GP priors, and later as the actual noise source driving the Langevin SDE.